

$L(2, 1)$ -coloring of the Fibonacci Cubes

Andrej Taranenko and Aleksander Vesel

University of Maribor, Faculty of Education
Koroška c. 160, SI-2000 Maribor, Slovenia
{andrej.taranenko|vesel}@uni-mb.si

Abstract. An $L(2, 1)$ -coloring of a graph G is an assignment of labels from $\{0, 1, \dots, \lambda\}$ to the vertices of G such that vertices at distance two get different labels and adjacent vertices get labels that are at least two apart. The λ -number $\lambda(G)$ of G is the minimum value λ such that G admits an $L(2, 1)$ -coloring. It is well known that the problem of determining the λ -number is NP-hard. The Fibonacci cube network was recently proposed as an alternative to the hypercube network. Three different evolutionary algorithms are presented to find optimal or near optimal $L(2, 1)$ -coloring of the Fibonacci cubes. The algorithms are compared with the Petford-Welsh probabilistic algorithm.

Keywords. $L(2, 1)$ -coloring, Fibonacci cube network, evolutionary algorithm.

1 Introduction

Consider the problem of assigning frequencies to radio transmitters at various nodes in a territory. Transmitters that are close must receive frequencies that are sufficiently apart, for otherwise they may be at the risk of interfering with each other. The spectrum of frequencies is a very important resource on which there are increasing demands, both civil and military. This calls for an efficient management of the spectrum. It is assumed that transmitters are all of identical type and that signal propagation is isotropic. Further, since frequencies themselves are quantized in practice, there is no loss of generality in assuming that they admit integer values.

The foregoing problem, with the objective of minimizing the span of frequencies, was

first placed on a graph-theoretical footing in 1980 by Hale [7] who established its equivalence to generalized vertex coloring problem that is known to be computationally hard. (Vertices correspond to transmitter locations and their labels to radio frequencies, while adjacencies are determined by geographical "proximity" of the transmitters.) Roberts [12] subsequently proposed a variation to the problem in which distinction is made between transmitters that are "close" and those that are "very close." This enabled Griggs and Yeh [6] to formulate the $L(2, 1)$ -coloring of graphs that has since been an object of extensive research, see for instance [9] and the references cited therein.

The topology of the communication network often influences the effectiveness of a parallel or distributed system. Hypercube networks with their extensive range of properties have appealed much attention in recent years and have been extensively investigated. Routing in a hypercube is a simple function of the Hamming distance between two nodes. That is, the message is successively sent along the connection corresponding to the bit position in both binary representations of the nodes with different values.

The Fibonacci cube is a communication network that possesses many suitable properties that are important in network design and application. Its major advantage is that it uses fewer links than the comparable hypercube, while its size does not increase as fast as the hypercube's. In other words, they allow more alternatives to build networks of various sizes. Note also that the Fibonacci cube can emulate many hypercube algorithms with efficiency [8]. Moreover, they emulate other

topologies, such as trees, rings and meshes very efficiently [14]. The $L(2,1)$ -coloring for some of these realistic topologies for wireless networks has been widely studied in the past, e.g. see [2] and the references therein.

It is well known that the problem of determining the λ -number is NP-hard. The problem remains NP-hard even when restricted to some classes of graphs: planar graphs, bipartite graphs, chordal graphs, and split graphs [3]. However, efficient solution exists with respect to λ -number of a tree or a k -almost tree where k is fixed [4]. For additional information relating to complexity issues, we refer to [3, 4].

Hypercubes Q_k are among the graphs that have been very extensively studied [13]. On the other hand we are not aware of any results concerning the λ -number of the Fibonacci cubes. As far as we know NP-hardness of the $L(2,1)$ -coloring problem has yet been proved neither for the hypercubes nor for the Fibonacci cubes. However, despite the extensive studies only the upper and the lower bounds on λ -number for the hypercubes are known. The lower bound equals $k+3$ and the most recent upper bound [13] is in the range $\lfloor [k+1+\log k], 2k \rfloor$. This big gap between the upper and the lower bound suggests the use of approximative algorithms in order to obtain better results. Moreover, the hardness of the problem with regard to the hypercubes arises a natural question of determining the λ -number of the Fibonacci cubes. As the first step toward the answer of this question we report on results obtained with evolutionary and probabilistic algorithms.

This paper is structured as follows: Section 2 presents some basic definitions concerning the Fibonacci cubes, as well as some basic properties of the channel assignment with separation. Section 3 presents four heuristic (probabilistic) optimization methods: the Petford-Welsh and 3 variants of the evolutionary algorithms for the search of $L(2,1)$ -coloring in the Fibonacci cubes. Lastly, Section 4 concludes with the results obtained from the computational implementations from the proposed heuristic methods.

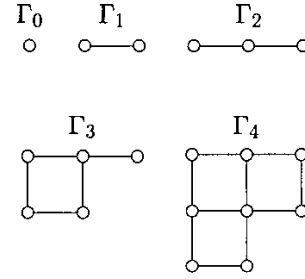


Figure 1: The Fibonacci cubes for the first values of n

2 Preliminaries

The *Fibonacci numbers* form a sequence of positive integers F_n , where $F_0 = 1$; $F_1 = 2$ and every other integer satisfies the recurrence $F_{n+2} = F_{n+1} + F_n$. It is known that any natural number can be uniquely represented as a sum of Fibonacci numbers (Zeckendorf's Theorem). Assume that i is a positive integer such that $i \leq F_{n-1} - 1$. Let $F(i) := b_{n-1}, \dots, b_1, b_0$ denote the order- n Fibonacci string of i , where $i = \sum_{j=0}^{n-1} b_j F_j$ and b_j is either 0 or 1, $0 \leq j \leq n-1$, with the condition $b_j b_{j+1} = 0$.

The *Hamming distance* between two binary strings x and y denoted $H(x, y)$ is the number of bits where x and y differ.

The *hypercube* of order n and denoted Q_n is the graph (V, E) where the vertex set V is the set of all binary strings $b_{n-1}, \dots, b_1, b_0$. Two vertices $x, y \in V$ are adjacent in Q_n if and only if $H(x, y) = 1$.

The *Fibonacci cube* denoted Γ_n is for $n \geq 0$ defined as follows. The vertex set of Γ_n is the set $V = \{0, 1, \dots, F_{n-1}\}$. Two vertices $x, y \in V$ are adjacent in Γ_n if and only if $H(F(x), F(y)) = 1$.

In [8] it was proved that Γ_{n+2} can be decomposed into Γ_{n+1} and Γ_n . The two subgraphs are disjoint and are connected exactly by F_n edges.

The Fibonacci cubes Γ_n for $n = 0, 1, 2, 3, 4$ are illustrated on Fig. 1.

Let $G = (V, E)$ be a graph with the vertex set $V(G)$ and the set of edges $E(G)$. A sequence of different vertices $u = x_0, x_1, x_2, \dots, x_{n-1}, x_n = v$, where $x_i x_{i+1}$ is an edge in G , is called a *path* and denoted P_{uv} . The number of edges in the path P_{uv} is

called the *length* of the path P_{uv} . The distance between vertices u and v is denoted $d(u, v)$ and defined as the length of the shortest path between u and v .

A $L(2, 1)$ -coloring of a graph G is a function f from the vertex set $V(G)$ to the set of non-negative integers C (called *colors*) such that

$$\begin{aligned} |f(u) - f(v)| &\geq 2 \text{ if } d(u, v) = 1, \text{ and} \\ |f(u) - f(v)| &\geq 1 \text{ if } d(u, v) = 2. \end{aligned}$$

An k - $L(2, 1)$ -coloring is a $L(2, 1)$ -coloring of G such that $C = \{0, \dots, k\}$. An optimal $L(2, 1)$ -coloring of G is a k - $L(2, 1)$ -coloring with k smallest possible. The largest color used by an optimal $L(2, 1)$ -coloring is called the λ -number of G and denoted by $\lambda(G)$.

3 Heuristic optimization

In this section we describe the methods we have used for finding the λ -number of Fibonacci cubes. We first present the probabilistic Petford-Welsh algorithm and then three versions of evolutionary algorithms: the evolutionary algorithm with permutational encoding of chromosomes, the evolutionary algorithm with integer solution representation and the hybrid evolutionary algorithm combined with the Petford-Welsh algorithm.

3.1 The Petford-Welsh algorithm

The Petford-Welsh algorithm [11] and some of its generalizations have proved to be a reasonably good heuristics for coloring various types of graphs including random k -colorable graphs, DIMACS challenge graphs, frequency assignment “realistic” graphs, and others. Moreover, it can be easily adjusted for other coloring-type problems. Basic steps of the algorithm are shown in Algorithm 1.

The greedy coloring always takes the minimal color which does not violate any constraints. Eventually the algorithm finds a $L(2, 1)$ coloring with k colors for some k . We hence know that $\lambda(G) \leq k$. The role of the first line in the while loop is to reduce the number of colors in the case when a proper coloring was found. All vertices of color k are recolored to obtain a (usually not proper)

```

begin
  p = random permutation of graph's
  vertices
  use greedy algorithm over p
  while not meet stopping condition do
    if a good coloring then
      recolor vertices with the highest
      assigned color
    randomly select a bad vertex v
    assign a new color to vertex v
end

```

Algorithm 1: Petford-Welsh algorithm

coloring with at most $k - 1$ colors. The algorithm continues and tries to find a proper coloring with less than k colors.

In the second line of the while loop, a bad vertex or a *violator* (one of its neighboring vertices or one of its vertices at distance two is violating a proper $L(2, 1)$ -coloring) is selected uniformly at random among the bad vertices. A new color is assigned at random. The new color is taken from the set $\{0, 1, \dots, k\}$ by sampling according to distribution defined as follows: The probability of color i to be chosen as a new color of vertex v is proportional to

$$\exp(-S_i/T)$$

where S_i is the number of edges with one endpoint at v and violating the constraint provided color of vertex v is given color i .

T is a parameter of the algorithm, which may be called temperature because of the analogy to temperature of the simulated annealing algorithm.

The stopping criteria we choose is a time limit (in terms of the number of calls to the function which computes a new color) and the number of colors with which we want to color the graph. The algorithm stops if either of the two conditions is satisfied.

3.2 Permutation representation of evolutionary algorithm - pEA

Main steps of the evolutionary algorithms we present herein are outlined in Algorithm 2.

The solutions are presented as permutations of graph's vertices. Function Create

```

begin
  Create( $P_{nuGen}$ )
  Evaluate( $P_{nuGen}$ )
  while ( $nuGen < MaxGenNum$ ) do
     $P_{nuGen+1} = Selection(P_{nuGen})$ 
    Crossover( $P_{nuGen+1}$ )
    Mutation( $P_{nuGen+1}$ )
    Evaluate( $P_{nuGen+1}$ )
     $nuGen = nuGen + 1$ 
  end
end

```

Algorithm 2: Evolution cycle

randomly initializes the population of chromosomes. Function **Evaluate** returns the largest used color assigned to the permutation by the greedy algorithm. An extra fitness evaluation function is used, defined by $f(x) = \exp(MaxColor - Col + 1)$, where $MaxColor$ denotes the largest color used in the whole population, and Col denotes the largest color assigned to permutation x by the greedy algorithm. The fitness function allows the algorithm to emphasize the difference between similarly good solutions. The pEA evolutionary algorithm uses 2-tournament selection. The stopping condition is the maximum number of generations denoted $MaxGenNum$.

The algorithm uses order crossover - OX. Other crossover operators, partially mapped crossover and cycle crossover, have also been tested [1, 10]. However, the best results had been obtained with the OX operator. The mutation operator simply exchanges two values in the chromosome.

Determining the proper values for the crossover probability p_c and the probability of mutation p_m is a crucial part of using evolutionary algorithms. To determine these values we have performed a series of tests. These were set up to increase the values of p_c with the step 0.05 in the range $[0, 1]$, and for each of these values the same procedure was used on p_m . The pair (p_c, p_m) where the best results were obtained was chosen. The optimal values determined were $p_c = 0.75$ and $p_m = 0.1$.

3.3 Integer representation evolutionary algorithm - iEA

In this version of the evolutionary algorithm the chromosomes are presented as integer

strings of length n , where n is the number of vertices in a graph. The i -th value in the string represents the color assigned to the i -th vertex. The basic steps are very similar to those presented with pEA.

The population is initialized with the help of the greedy algorithm, where a random permutation is formed and the coloring assigned to it represents a chromosome. Let k_0 denote the largest color used in the whole population. In this case the largest allowed color in evolution process is set to $k = k_0 - 1$.

Function **Evaluate** counts the number of bad vertices in a given coloring (chromosome) of the population. The aim is to find a proper $L(2, 1)$ -coloring, ie. a coloring where the number of violators equals 0. The iEA algorithm also uses 2-tournament selection.

The evolution process stops when at least one of the stopping criteria is met: (i) the algorithm found a proper $k - L(2, 1)$ -coloring or (ii) the maximum number of generations is exceeded. If case (i) occurs, the largest allowed color is decreased by one. All solutions in the population are then "fixed" by decreasing the color of all vertices with the color greater than the maximum allowed color. The new population is evaluated and then the evolution continues.

The crossover operator used is a problem specific operator, originally applied for ordinary graph colorings [5]. The operator forms an offspring from two parent chromosomes by coloring the vertices, which are violators in one parent, with the color of the parent where the same vertices are not violators. In the case where a vertex is violating the rules of proper coloring in both parents, the least frequent color among the neighboring vertices and the vertices at the distance two is used. All the remaining vertices are randomly colored with a color from one of the parents. This crossover operator tries to reduce the number of violators in a coloring.

The mutation in iEA randomly chooses a violator in the chromosome and assigns a new color to the chosen vertex. The probability of a violator being chosen is the same for all violators. If there does not exist a violator in the given chromosome, the mutation is not performed.

The probabilities for crossover and mutation have been determined by testing as described in the previous section and finally set to $p_c = 0.3$ and $p_m = 0.9$.

3.4 Hybrid evolutionary algorithm

In hybrid evolutionary algorithms mutation is usually replaced by a local search heuristics in order to obtain better results. In the presented version of the hybrid algorithm (iEA-PW) mutation was replaced by an adopted version of the Petford-Welsh algorithm presented above.

The solutions are presented as integer strings and the main steps are similar to those in iEA.

There are two decisions which are of crucial importance for the behavior of the Petford-Welsh algorithm: to define the number of iterations before restart (or define a stopping criteria) and to define a good value for parameter T . We have used a constant function $repeat = 10^5$ that suffices for all graphs of the interest. The value of parameter T was determined by testing different values. The final value we ended up with was $T = 0.23$. The choices we made work well on instances tested, but in general fine tuning is probably needed as is the usual case with other heuristics of similar type such as simulated annealing, tabu search, etc.

4 Results and conclusions

The algorithms have been tested on the Fibonacci cubes Γ_n with up to 2584 vertices ($n \leq 16$). Table 1 shows the best results obtained with specific algorithms as seen in the heading of the table. Results in boldface mean that no other algorithm achieved the same nor a better result.

The parameters for the evolutionary algorithms were as follows. The maximum allowed number of generations was set to $MaxGenNum = 2000$, the size of a population was $PopSize = 40$. The probability of crossover and mutation in iEA were $p_c = 0.3$ and $p_m = 0.9$, respectively, whereas they were set to $p_c = 0.75$ and $p_m = 0.1$ in pEA version of the evolutionary algorithm. The tempera-

Table 1: Best results for specific algorithms

graph	n	pEA	iEA	iEA-PW	PW
Γ_2	3	3	3	3	3
Γ_3	5	4	4	4	4
Γ_4	8	5	5	5	5
Γ_5	13	6	7	6	6
Γ_6	21	7	8	7	8
Γ_7	34	9	9	8	9
Γ_8	55	10	10	10	10
Γ_9	89	13	12	11	12
Γ_{10}	144	15	13	13	13
Γ_{11}	233	17	15	14	15
Γ_{12}	377	20	18	16	17
Γ_{13}	610	22	20	18	18
Γ_{14}	987	25	22	20	20
Γ_{15}	1597	27	25	22	22
Γ_{16}	2584	30	28	24	24

Table 2: Statistics for iEA-PW

graph	n	λ_{min}	λ_{avg}	σ
Γ_2	3	3	3	0
Γ_3	5	4	4	0
Γ_4	8	5	5	0
Γ_5	13	6	6,5	0,53
Γ_6	21	7	7,6	0,52
Γ_7	34	8	9	0,47
Γ_8	55	10	10,1	0,32
Γ_9	89	11	11,9	0,74
Γ_{10}	144	13	13,4	0,52
Γ_{11}	233	14	15,1	0,57
Γ_{12}	377	16	16,9	0,57
Γ_{13}	610	18	18,7	0,67
Γ_{14}	987	20	20,5	0,53
Γ_{15}	1597	22	22,6	0,52
Γ_{16}	2584	24	24,5	0,53

ture in the Petford-Welsh algorithm was set to $T = 0.23$ in both - the hybrid evolutionary algorithms as well as in the stand alone version. The probability of mutation in the hybrid algorithm was set to $p_m = 2/n$.

Tables 2, 3 and 4 show some statistics of the hybrid, iEA and pEA evolutionary algorithms. The column λ_{min} shows the best found result, the average of ten tests is given in the column λ_{avg} and the standard deviation of the data is represented in the column marked with σ .

We have presented four methods for find-

Table 3: Statistics for pEA

graph	n	λ_{min}	λ_{avg}	σ
Γ_2	3	3	3	0
Γ_3	5	4	4	0
Γ_4	8	5	5	0
Γ_5	13	6	6	0
Γ_6	21	7	7	0
Γ_7	34	9	9	0
Γ_8	55	10	10,7	0,48
Γ_9	89	13	13	0
Γ_{10}	144	15	15	0
Γ_{11}	233	17	17,4	0,52
Γ_{12}	377	20	20	0
Γ_{13}	610	22	22	0
Γ_{14}	987	25	25	0
Γ_{15}	1597	27	27,7	0,48
Γ_{16}	2584	30	30,2	0,42

Table 4: Statistics for iEA

graph	n	λ_{min}	λ_{avg}	σ
Γ_2	3	3	3	0
Γ_3	5	4	4	0
Γ_4	8	5	5	0
Γ_5	13	7	7	0
Γ_6	21	8	8	0
Γ_7	34	9	9	0
Γ_8	55	10	10,5	0,53
Γ_9	89	12	12	0
Γ_{10}	144	13	13,6	0,52
Γ_{11}	233	15	15,6	0,52
Γ_{12}	377	18	18	0
Γ_{13}	610	20	20,3	0,48
Γ_{14}	987	22	23	0,47
Γ_{15}	1597	25	25,7	0,48
Γ_{16}	2584	28	28,4	0,52

ing the λ -number of Fibonacci cubes: the probabilistic Petford-Welsh algorithm and three versions of evolutionary algorithms.

The data presented in Table 1 shows that the best results have been obtained with the hybrid evolutionary algorithm which is for all graphs of interest at least as effective as any other presented algorithm. In five cases the hybrid EA outperforms the stand alone Petford-Welsh implementation which is the runner-up of the comparison. Moreover, the algorithm has found the λ -numbers for the Fibonacci cubes Γ_n , where $n \leq 7$. These num-

bers have been confirmed by a program based on a simple backtracking algorithm.

References

- [1] Bäck T, Fogel D, Michalewicz Z. *Handbook of Evolutionary Computations*. Oxford: IOP Publishing Ltd and Oxford University Press; 1997.
- [2] Bertossi AA, Pinotti MC, Tan RB. *Channel assignment with separation for interference avoidance in wireless networks*. IEEE Trans. Parallel Distributed Systems 2003; 14 (3): 222–235.
- [3] H.L. Bodlaender, T. Kloks, R.B. Tan and J. van Leeuwen, λ -coloring of graphs (in: H. Reichel and S. Tison, Eds., STACS 2000, Lille, France, February 2000) Lect. Notes Comput. Sci. 2000; 1770: 395–406.
- [4] J. Fiala, T. Kloks and J. Kratochvíl, *Fixed-parameter complexity of λ -labelings*. Discrete Appl. Math. 2001; 113: 59–72.
- [5] Fleurent C, Ferland JA. *Genetic and Hybrid Algorithms for Graph Coloring*. Annals of Operations Research 1996; 63: 437–461.
- [6] Griggs JR, Yeh RK. *Labeling graphs with a condition at distance two*. SIAM Journal of Discrete Mathematics 1992; 5: 586–595.
- [7] Hale WK. *Frequency assignment: Theory and application*. Proc. IEEE 1980; 68: 1497–1514.
- [8] Hsu WJ. *Fibonacci cubes - a new interconnection topology*. IEEE Trans. Parallel Distributed Systems 1993; 4 (1): 3–12.
- [9] Klavžar S, Vesel A. *Computing graph invariants on rotagraphs using dynamic algorithm approach: the case of (2,1)-colorings and independence numbers*. Discrete Applied Mathematics 2003; 129: 449–460.
- [10] Michalewicz Z. *Genetic Algorithms + Data Structures = Evolution Programs*. Berlin: Springer-Verlag; 1999.
- [11] Petford AD, Welsh DJA. *A randomized 3-colouring algorithm*. Discrete Mathematics 1989; 74(1): 253–261.
- [12] Roberts FS. *T-colorings of graphs: Recent results and open problems*. Discrete Mathematics 1991; 93: 229–245.
- [13] Whittlesey MA, Georges JP, Mauro DW. *On the λ -number of Q_n and related graphs*. SIAM Journal Discrete. Mathematics 1995; 8: 499–506.
- [14] Wasserman HC., Ghazati SA. *Generalized linear recursive networks: topological and routing properties*. Computers & Electrical Engineering 2003; 29: 121–134.